

Worked for 12s

You are not lost because the project is impossible; you are lost because it currently combines **quantum physics, sensing, machine learning, hardware security, and statistical detection** in one description. The right way to understand it is to separate those layers and rebuild the idea one step at a time.

Tutorial: Adversarial Quantum Sensing

Step 1: State the project in one sentence

We want to determine whether an attacker can manipulate the electronic controls of an NV quantum magnetometer so that it reports a false magnetic field, and whether randomized sensor controls can prevent or reveal the attack.

The project has three central questions:

1. Can the sensor be spoofed?
2. Which estimators are most affected?
3. Can randomized control settings protect the sensor?

Everything else supports one of these questions.

Step 2: Understand what the sensor does

A magnetometer estimates a magnetic field B .

A conventional magnetometer converts the field directly into a voltage, current, frequency, or another classical signal.

An NV magnetometer uses the quantum state of an electron spin inside a nitrogen-vacancy defect in diamond. The magnetic field changes the phase of this spin. We control the spin with microwave pulses and read it optically.

The basic signal path is:



The sensor does not directly print the magnetic field. It produces measurement outcomes from which an estimator infers the field.

Step 3: Understand the NV center as a qubit

An NV center has several energy levels, but the initial simulator uses two selected levels:

$$|0\rangle, \quad |1\rangle.$$

We treat these levels as a qubit.

The magnetic field affects the relative phase between them. If the spin starts in a superposition,

$$|\psi_0\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}},$$

then after sensing the field for time t , its state becomes approximately

$$|\psi(B, t)\rangle = \frac{|0\rangle + e^{i\gamma B t} |1\rangle}{\sqrt{2}}.$$

Here:

- B is the magnetic field.
- t is the sensing time.
- γ is the electron gyromagnetic ratio.
- $\gamma B t$ is the accumulated phase.

Define the magnetic phase as

$$\phi_B = \gamma B t.$$

This is the physical quantity the experiment observes indirectly.

Step 4: Understand Ramsey sensing

A Ramsey experiment has three main operations.

4.1 Prepare the spin

The NV spin starts in $|0\rangle$. A microwave $\pi/2$ pulse creates a superposition:

$$|0\rangle \longrightarrow \frac{|0\rangle + |1\rangle}{\sqrt{2}}.$$

4.2 Allow the spin to sense the field

The spin evolves freely for time t . The magnetic field creates a relative phase:

$$\phi_B = \gamma Bt.$$

4.3 Convert phase into population

A second $\pi/2$ pulse converts the accumulated phase into a probability that can be measured.

The entire process is:



The second pulse has a controllable analysis phase α . This lets us observe different components of the quantum state.

Step 5: Convert the quantum state into a measurable probability

In the simplified model, the probability of observing outcome 0 is

$$p_0(B, t, \alpha) = \frac{1}{2} [1 + v(t) \cos(\gamma Bt - \alpha)].$$

The visibility term is

$$v(t) = (1 - 2\eta)C(t),$$

where

$$C(t) = \exp \left[- \left(\frac{t}{T_2^*} \right)^p \right].$$

The parameters mean:

Parameter	Meaning
B	True magnetic field
t	Sensing duration
α	Analysis-pulse phase
γ	Electron gyromagnetic ratio
T_2^*	Ramsey coherence time
p	Dephasing exponent
η	Readout-error probability
$C(t)$	Remaining coherence

Longer sensing time creates more phase, which can improve sensitivity. However, longer times also cause more decoherence.

This produces a natural tradeoff:

more phase accumulation versus less remaining coherence.

Step 6: Understand why repeated measurements are necessary

One execution produces only one binary outcome:

$$y \in \{0, 1\}.$$

A single outcome tells us very little. We repeat the experiment many times.

If the probability of observing zero is p_0 , then the number of zero outcomes from N repetitions is

$$n_0 \sim \text{Binomial}(N, p_0).$$

For example, if we perform 1,000 measurements and observe 700 zero outcomes, the empirical probability is

$$\hat{p}_0 = \frac{700}{1000} = 0.7.$$

The estimator uses these observed probabilities to infer B .

In the initial project, the same simulated sensor performs measurements at several sensing times:

$$t_k \in \{5, 10, 20, 40, 60\} \mu s.$$

This is still **one sensor**. It is simply operated under five different settings.

Step 7: Understand why we use multiple sensing times

The Ramsey probability is periodic because it contains a cosine:

$$\cos(\gamma B t - \alpha).$$

Two different magnetic fields can sometimes produce the same probability at one sensing time. This is phase ambiguity or phase wrapping.

Measurements at several sensing times help distinguish these fields.

For every field B_i , the simulator produces a sensing trajectory:

$$R_i = \{\rho(B_i, t_1), \rho(B_i, t_2), \dots, \rho(B_i, t_M)\}.$$

For a measurement based estimator, the trajectory might instead contain estimated expectation values or observed probabilities.

Step 8: Understand the field estimator

The sensor produces measurements. An estimator converts them into

$$\hat{B}.$$

We will compare several estimators because they receive different amounts of information.

Estimator	Input
Maximum likelihood	Ramsey measurement outcomes and known settings
Linear regression	Measured features
RBF kernel regression	Measured features
Neural network	Measured features

Estimator	Input
Tomography kernel	Reconstructed density matrices
Exact-state kernel	Exact simulated density matrices

The maximum-likelihood estimator is the physics baseline. It asks:

Which magnetic field would most likely have produced these measurements?

The learning models ask:

Based on training examples, which magnetic field is associated with this pattern?

Step 9: Understand the role of QML

The original research question was whether preserving quantum-state information improves field estimation.

A measurement based model receives reduced information such as

$$\langle X \rangle, \quad \langle Y \rangle, \quad \langle Z \rangle,$$

possibly corrupted by finite-shot and readout noise.

The exact-state model receives the simulated density matrix before measurement:

$$\rho(B, t).$$

It can construct a similarity kernel such as

$$K_{ij} = \frac{1}{M} \sum_{k=1}^M \text{Tr} [\rho(B_i, t_k) \rho(B_j, t_k)].$$

The exact-state kernel is an **oracle upper bound**. A real sensor does not reveal its exact density matrix. It is useful because it tells us how much information could theoretically be available before measurement.

The new security project asks:

If coherent quantum information improves estimation, does manipulating that information also produce different security behavior?

We do not know the answer yet. That is one of the experiments.

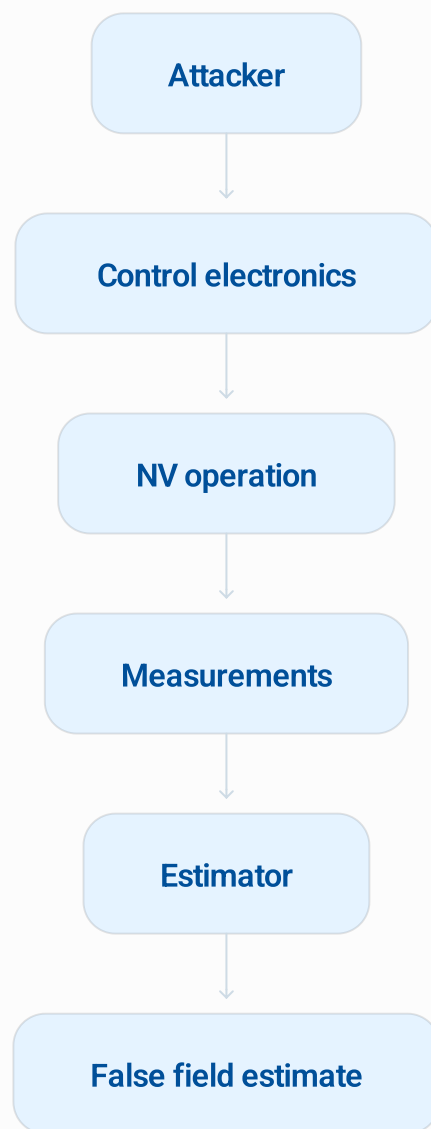
Step 10: Introduce the attacker

The NV spin is controlled by classical electronics:

- DDS phase registers
- Microwave generators
- I/Q modulators
- AWG timing controllers
- FPGA pulse sequencers

The attacker does not need to alter the physical magnetic field. The attacker can alter the controls used to interrogate the sensor.

The attack path becomes:



The true field remains B , but the estimator reports

$$\hat{B}_A \neq B.$$

For a targeted attack, the adversary selects a desired field B_T and tries to cause

$$\widehat{B}_A \approx B_T.$$

Step 11: Understand the constant phase attack

Suppose the attacker changes the analysis-pulse phase by $\delta\phi$:

$$\alpha \longrightarrow \alpha + \delta\phi.$$

The measured probability becomes

$$p_0^A = \frac{1}{2} [1 + v(t) \cos(\gamma B t - \alpha - \delta\phi)].$$

If the estimator does not know about the attack, it may interpret this phase shift as a different magnetic field.

For a single sensing time,

$$\Delta B \approx \frac{\delta\phi}{\gamma t}.$$

A constant phase offset therefore creates different apparent field errors at different sensing times:

$$\Delta B_k = \frac{\delta\phi}{\gamma t_k}.$$

That inconsistency may make a simple attack detectable.

Step 12: Understand the targeted coherent spoof

A knowledgeable attacker can make the added phase proportional to sensing time:

$$\delta\phi_k = \gamma \Delta B t_k.$$

The total phase becomes

$$\gamma B t_k + \delta\phi_k = \gamma (B + \Delta B) t_k.$$

The attacked sensor at field B now behaves like a clean sensor at field

$$B + \Delta B.$$

For example, suppose the true field is

$$B = 1.00 \mu\text{T},$$

and the attacker wants the system to report

$$B_T = 1.01 \mu\text{T}.$$

Then

$$\Delta B = 10 \text{ nT}.$$

The attacker calculates a different phase shift for every sensing time:

Sensing time	Required phase shift
$5 \mu\text{s}$	0.0088 rad
$10 \mu\text{s}$	0.0176 rad
$20 \mu\text{s}$	0.0352 rad
$40 \mu\text{s}$	0.0704 rad
$60 \mu\text{s}$	0.1057 rad

This produces a consistent apparent field displacement across all sensing times.

Step 13: Understand the fundamental detection problem

This is the most important concept in the project.

Consider two situations:

Situation A: Genuine field change

$$B \longrightarrow B + \Delta B.$$

Situation B: Control attack

$$B \longrightarrow B, \quad \delta\phi_k = \gamma\Delta B t_k.$$

Under the ideal model, both situations can produce the same measurement distribution:

$$P_{\text{attack}}(y \mid B) = P_{\text{clean}}(y \mid B + \Delta B).$$

If the detector sees only sensor measurements, it cannot determine which situation occurred.

This is an **identifiability limitation**. It is not a weakness in a particular machine-learning model. The available data contain no distinguishing evidence.

This control experiment is scientifically valuable. The simulator should deliberately demonstrate this equivalence.

Step 14: Distinguish state attacks from measurement attacks

This distinction prevents a major conceptual error.

Let

$$\rho_{\text{pre}}$$

be the quantum state immediately before the final analysis pulse.

If the attacker manipulates the **preparation pulse**, the attacker changes the state:

$$\rho_{\text{pre}} \longrightarrow \rho_{\text{pre}}^A.$$

If the attacker manipulates only the **final analysis pulse**, the preanalysis state remains unchanged. The attacker changes how the state is measured:

$$M_y \longrightarrow M_y^A.$$

The outcome probability becomes

$$p_A(y) = \text{Tr}(M_y^A \rho_{\text{pre}}).$$

Therefore:

Attack location	Preanalysis state changes?	Measurement changes?
Preparation pulse	Yes	Indirectly
Free evolution	Yes	Indirectly
Final analysis pulse	No	Yes
Readout electronics	No	Yes

A final-pulse attack cannot automatically distort a kernel computed from the state before that pulse. This must be represented correctly in the simulator.

Step 15: Understand the timing attack

The clean free-evolution time is t . The attacker changes it to

$$t_A = t + \delta t.$$

The phase becomes

$$\phi_A = \gamma B(t + \delta t).$$

If the estimator assumes the original time t , the approximate reported field is

$$\widehat{B}_A \approx B \left(1 + \frac{\delta t}{t} \right).$$

The approximate field error is

$$\Delta B_{\text{timing}} \approx B \frac{\delta t}{t}.$$

This attack has two special properties:

1. Its effect depends on the true field B .
2. It also changes the amount of dephasing because the actual evolution time changes.

At $B = 0$, the simple timing attack cannot create a nonzero magnetic phase. Phase or frequency-reference manipulation is more powerful for that objective.

A global delay applied to the entire experiment also does not matter for a static field. The attacker must change the separation between the two Ramsey pulses.

Step 16: Understand I/Q attacks

A microwave pulse can be described by

$$H_C(t) = \frac{\hbar}{2} [\Omega_I(t)\sigma_x + \Omega_Q(t)\sigma_y + \Delta\omega(t)\sigma_z].$$

The I and Q components determine the pulse's rotation axis and phase.

An attacker can change:

- I-channel gain
- Q-channel gain
- Relative I/Q phase
- Pulse amplitude
- Pulse duration
- Microwave detuning

This can change an ideal rotation

$$R_{\mathbf{n}}(\theta)$$

into

$$R_{\mathbf{n}'}(\theta + \delta\theta).$$

A fixed deterministic I/Q error is still a coherent rotation. It should not automatically reduce state purity. Random pulse-to-pulse errors can produce effective decoherence.

Step 17: Understand the proposed defense

A fixed sensing schedule is predictable:

$$t_k \in \{5, 10, 20, 40, 60\} \mu s.$$

An attacker who knows the schedule can calculate

$$\delta\phi_k = \gamma \Delta B t_k$$

before the experiment.

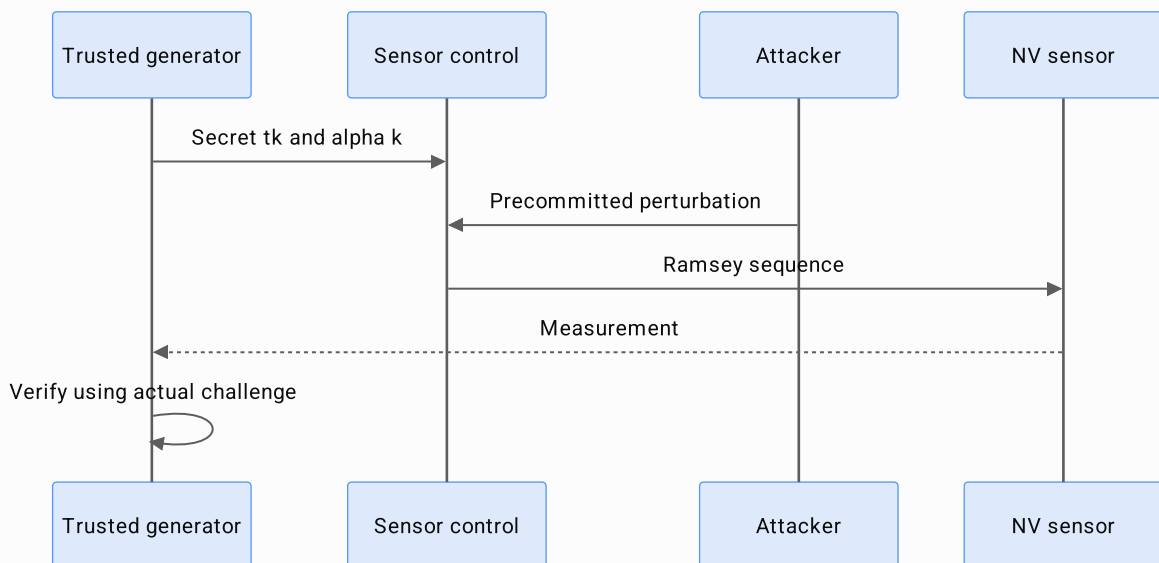
The proposed defense randomly selects the sensing settings:

$$q_k = (t_k, \alpha_k).$$

Here q_k is a secret challenge generated for measurement k .

The legitimate system knows the actual t_k and α_k . A precommitted attacker does not.

If the attacker does not know t_k , it cannot calculate the correct phase shift needed to imitate a constant field displacement.



The defense depends on timing:

- When is the challenge generated?

- When can the attacker observe it?
- When must the attacker commit its perturbation?
- Can the attacker alter the analog signal after the digital controls are verified?

Randomization works when the attacker learns the challenge too late to respond correctly.

Step 18: Understand why random analysis phase is insufficient by itself

Suppose the nominal analysis phase is α_k . An attacker adds a relative offset:

$$\alpha_k^A = \alpha_k + \delta\phi_k.$$

The attacker may not need to know the original value of α_k to add this offset.

Therefore, randomizing α_k alone does not guarantee protection against all phase attacks.

Random sensing time is more directly relevant because a targeted field spoof requires

$$\delta\phi_k \propto t_k.$$

Even random sensing time will fail if the attacker learns t_k quickly enough or manipulates the frequency reference in a way that naturally accumulates phase with elapsed time. That is why the latency experiment matters.

Step 19: Define the attacker levels

We need more than one attacker model because defense performance depends on attacker knowledge.

Attacker	Capability
A0	Applies a fixed perturbation without knowing the schedule
A1	Knows the complete fixed schedule
A2	Learns each random challenge after a delay
A3	Immediately learns the challenge and controls the downstream analog signal

Expected behavior:

- A constant phase attack from A0 should produce inconsistencies.

- A1 may implement an exact spoof against a fixed schedule.
- Randomization may stop A1.
- A2 lets us measure how security changes with latency.
- A3 may defeat the randomized defense.

The defense should therefore be described as conditional:

$$\text{Security} \iff \text{attacker response time} > \text{available commitment window}.$$

Step 20: Understand the verifier

The verifier receives:

- Measurement outcomes
- Protected sensing times
- Protected analysis phases
- Expected sensor model
- Optional control-register or waveform evidence

It first estimates the magnetic field from an estimation subset:

$$\hat{B} = \arg \max_B \sum_{k \in \mathcal{E}} \log P(y_k \mid B, t_k, \alpha_k).$$

It then tests separate verification measurements:

$$k \in \mathcal{V}.$$

A simple residual statistic can measure whether the verification outcomes agree with the estimated field and secret challenges.

The estimation and verification subsets should be separate. Otherwise, the same observations are being used both to fit the field and evaluate whether the fitted field explains them.

The verifier produces:

$$(\hat{B}, s_A),$$

where s_A is an attack score.

Step 21: Understand what the simulator represents

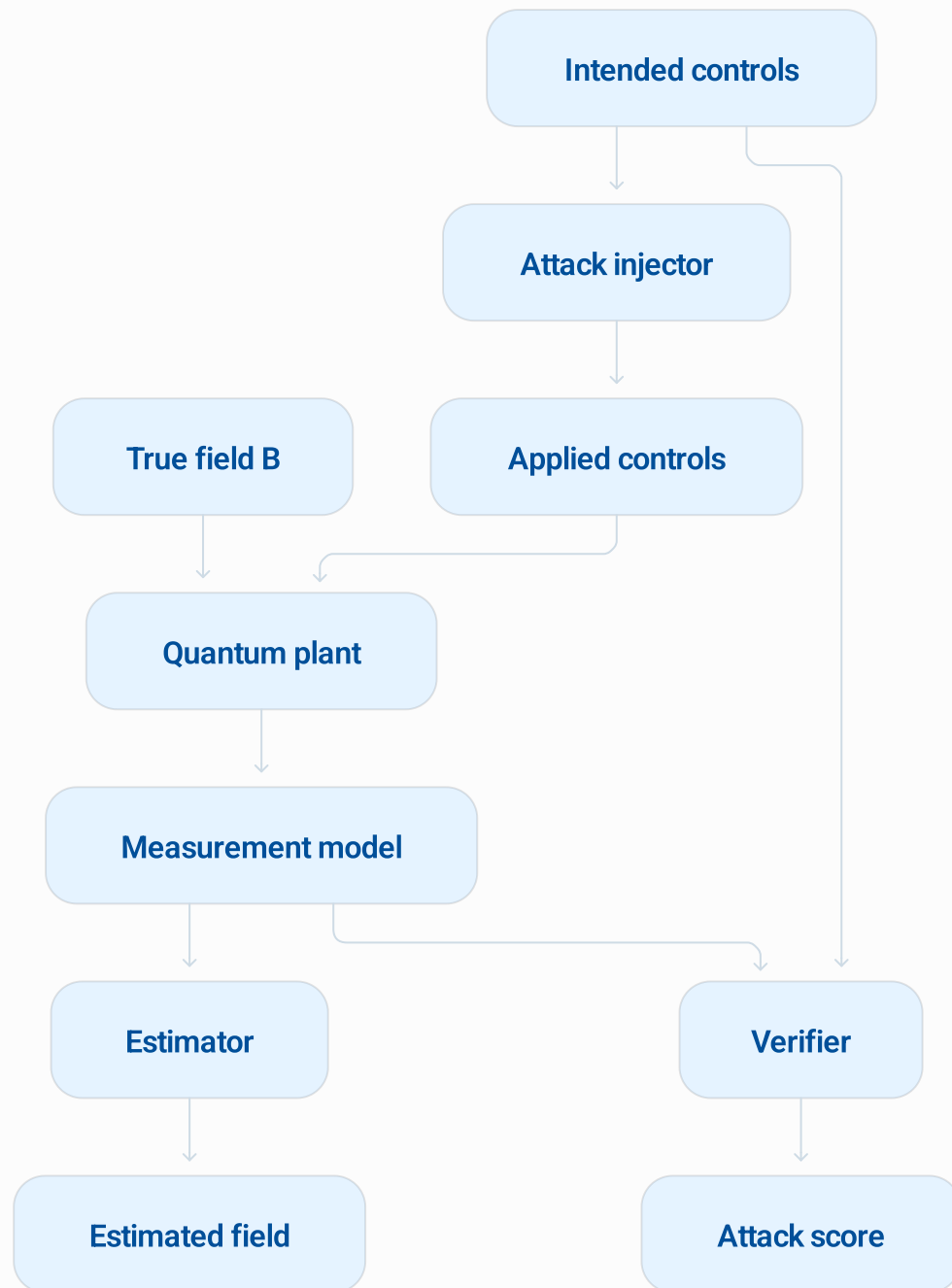
The simulator is a controlled virtual laboratory.

It contains six distinct versions of reality:

Object	Meaning
Ground truth	Actual field and environmental parameters
Intended controls	Commands requested by the trusted experiment
Attacked controls	Commands after adversarial modification
Physical state	Quantum state produced by the applied operations
Observations	Finite-shot measurement outcomes
Reported result	Estimated field and attack decision

Keeping these objects separate prevents hidden information leakage and incorrect security claims.

The complete data flow is:



Step 22: Build the simulator in layers

Layer 1: Analytical probability model

Compute p_0 directly from the Ramsey equation.

This layer is fast and easy to verify. Use it for large parameter sweeps.

Layer 2: Density matrix model

Explicitly calculate the quantum state after every operation.

This layer supports:

- Fidelity

- Trace distance
- State visualization
- Separation of state and measurement attacks
- Quantum kernels

Layer 3: Pulse-level model

Represent finite microwave pulses with a time-dependent Hamiltonian and master-equation evolution.

This layer supports realistic:

- Amplitude errors
- I/Q imbalance
- Pulse duration errors
- Detuning
- Decoherence during pulses

We should trust Layer 3 only after it agrees with Layers 1 and 2 in their shared ideal limit.

Step 23: Perform the clean baseline experiment

Before introducing an attacker, verify that the simulated sensor works.

For each true field:

1. Generate measurements at several sensing times.
2. Estimate the field.
3. Compare $\hat{B}$ with B .
4. Repeat using several random seeds.
5. Calculate RMSE, MAE, and bias.

The first plot should be:

$$B \text{ versus } \hat{B}.$$

A correct estimator should approximately follow

$$\hat{B} = B.$$

The second plot should show Ramsey probability versus sensing time or field. It should display oscillation with a decaying contrast envelope.

Step 24: Perform the first attack experiment

Start with a constant phase attack.

For every phase offset:

$$\delta\phi \in \{0, 0.001, 0.005, 0.01, \dots, 0.1\} \text{ rad},$$

run the same fields and sensing times.

Measure:

$$E_A = |\widehat{B}_A - B|.$$

Plot:

$$|\delta\phi| \quad \text{versus} \quad |\widehat{B}_A - B|.$$

This verifies that a control perturbation propagates into field-estimation error.

Step 25: Perform the exact-spoof experiment

Select several target displacements:

$$\Delta B \in \{1, 5, 10, 25, 50\} \text{ nT}.$$

For every sensing time, calculate

$$\delta\phi_k = \gamma \Delta B t_k.$$

Compare:

1. Clean data generated at $B + \Delta B$.
2. Attacked data generated at B .

Check whether their ideal probabilities agree:

$$\max_k |p_{\text{attack},k}(B) - p_{\text{clean},k}(B + \Delta B)| \approx 0.$$

This is the project's most important negative control.

Step 26: Perform the randomized-defense experiment

Compare three schedules:

1. Fixed t_k
2. Random t_k
3. Random t_k and α_k

Test each schedule against:

- A0 fixed attacker
- A1 schedule-aware attacker
- A2 delayed attacker
- A3 immediate attacker

Measure:

- Field-estimation error
- Targeted attack success rate
- Detection probability
- False-alarm probability
- Clean accuracy penalty

Then vary attacker latency. This gives a security curve:

attacker latency versus attack success rate.

This curve will tell us the hardware timing requirement for the proposed defense.

Step 27: Compare the estimators fairly

Every estimator should be tested on the same true fields, attacks, noise conditions, and random seeds.

However, their resource use differs.

A Z-only estimator receives fewer measurements than an XYZ or tomography estimator. The exact-state oracle receives information unavailable from physical finite-shot measurements.

Report three comparison modes:

Comparison	Purpose
Same generated samples	Controlled algorithm comparison
Same total shot budget	Experimental resource comparison
Oracle state access	Theoretical information upper bound

This prevents the oracle model from appearing experimentally superior simply because it receives perfect information.

Step 28: Define project success

The proof of concept succeeds if we can demonstrate the following chain:

realistic control manipulation $\rightarrow$ state or measurement change $\rightarrow$ estimator error

The initial success criteria are:

1. The analytical and density matrix models agree.
2. The clean sensor estimates the field with known error.
3. A constant phase attack causes measurable bias.
4. A schedule-aware attack produces a selected field displacement.
5. The simulator demonstrates the perfect-spoof identifiability limitation.
6. Random sensing time reduces attack success for at least one clearly defined attacker.
7. The latency experiment identifies where the defense fails.
8. Classical, tomography, and oracle estimators are compared under common attacks.
9. Every result is reproducible from a stored configuration and seed.

The project can still produce a valuable result if QML models show no special vulnerability. That would mean control-layer manipulation dominates estimator choice.

Step 29: Know what we are not claiming yet

The initial simulator will not prove that:

- A real NV experiment behaves exactly like the two-level model.
- Exact quantum states are available to a deployed estimator.
- Randomization stops an attacker with immediate downstream control.
- QML is universally more secure or less secure.
- A sensor network is necessary.
- The defense works without a protected trust boundary.

These questions belong to later hardware-calibrated and experimental stages.

Step 30: Follow the practical implementation order

The clearest development sequence is:

1. Implement units and validated configuration.
2. Implement the clean analytical Ramsey probability.
3. Implement finite-shot sampling.
4. Implement a maximum-likelihood field estimator.
5. Plot the clean sensing response.
6. Implement the density matrix version.

7. Verify agreement between the two sensor models.
8. Create separate intended and attacked control records.
9. Implement a constant analysis-phase attack.
10. Implement the targeted schedule-aware spoof.
11. Demonstrate the identifiability limitation.
12. Add classical learning estimators.
13. Add tomography and exact-state kernels.
14. Implement randomized sensing times.
15. Implement the verifier.
16. Sweep attacker knowledge and latency.
17. Add timing and I/Q attacks.
18. Add the pulse-level model.
19. Add hardware-calibrated parameters.
20. Consider a trusted reference sensor or small network.

The first runnable version only needs Steps 1–11. At that point, we will already have a coherent research result: a functioning sensor simulation, a physical attack, an estimator, and a formal demonstration of when the attack cannot be detected from sensor output alone.